

AIFP Improvement Proposal (AIP) Process

Document: AIFP-DOC-06 · **Version:** 1.0.0 · **Governed by:** AIFP-1 (Doc 01)

Status: Active · Modeled on Ethereum EIPs, Python PEPs, and the IETF RFC process.

An **AIP** (AiFinPay Improvement Proposal) is the formal mechanism for proposing, debating, and ratifying changes to the AiFinPay Paywall Protocol (AIFP) and its ecosystem. AIP-1 (this process) is itself an AIP and may only be changed by an AIP.

1. Purpose

The AIP process exists to make protocol evolution **transparent, reviewable, and backward-compatibility-aware**. It ensures that any change to AIFP-1, its APIs, schemas, SDKs, or governance is documented, discussed in the open, and ratified by a defined body before it becomes normative.

2. AIP Types

Type	Scope	Examples
Meta	Changes to processes, governance, or the AIP process itself.	Amending this document; changing the review board.
Informational	Guidelines, best practices, design notes; non-binding.	Recommended retry tuning; security advisories.

Standards Track AIPs are further sub-categorized: **Core** (protocol semantics), **Interface** (APIs/SDKs/schemas), **Networks** (chain support), **Security**.

3. Proposal Lifecycle



Status	Meaning	Entry criteria
Review	Under active community + editor review.	Editor marks ready; discussion thread open.
Last Call	Final 14-day window for objections.	Review complete; no unresolved blocking issues.
Accepted	Approved by the review board; awaiting reference implementation.	Board sign-off after Last Call.
Final	Normative. Part of the protocol.	Reference implementation merged + conformance tests pass.
Rejected	Declined with recorded rationale.	Board decision or author abandonment after review.
Withdrawn	Author retracts.	Author request.
Stagnant	Inactive >6 months in Draft/Review.	Auto-marked; may be resurrected.
Superseded	Replaced by a newer Final AIP.	Successor reaches Final.

A Standards Track AIP ****cannot reach Final without a working reference implementation and passing conformance tests**** (Doc 14).

4. Governance Process

4.1. Roles.

- **Author(s)** — write and champion the AIP.
- **AIP Editors** — enforce formatting, assign numbers, manage status transitions

(process, not merit).

- **Review Board** — domain maintainers (protocol, security, SDK, networks) who evaluate technical merit and vote.

- **Community** — anyone may comment; substantive objections must be addressed.

4.2. Decision rule. Acceptance requires **rough consensus** of the Review Board with **no unresolved blocking security objection**. Security-categorized AIPs require explicit sign-off from the Security maintainer. Ties default to the **status quo** (reject).

4.3. Transparency. All discussion happens in public (GitHub Discussions / PRs).

Decisions are recorded in the AIP header (`Resolution:` link).

5. Versioning & Backward Compatibility

AIFP follows **semantic versioning** (`MAJOR.MINOR.PATCH`) at the protocol level:

- **PATCH** — editorial/clarification; no wire change.

- **MINOR** — backward-compatible additions (new optional field, new error code, new

network, new endpoint). Existing clients keep working.

- **MAJOR** — breaking change. Requires a migration AIP, a deprecation window, and

parallel support of the previous MAJOR for **≥12 months**.

Compatibility rules (normative for any AIP):

1. New required request fields are a **breaking** change (MAJOR).
2. New optional fields and new response fields are **non-breaking** (MINOR).
3. Removing or renaming any field/endpoint/error is **breaking** (MAJOR).
4. Receipt claim changes must preserve verification of previously issued receipts within

their TTL.

1. The **Protocol Negotiation Layer** (AIFP-1 §22.3) **MUST** be used to gate any

capability that isn't universally available.

6. Reference Implementation Rules

- Every Standards Track AIP **MUST** link a reference implementation before **Final**.
- The implementation **MUST** pass the AIFP **conformance test suite** (Doc 14) and update it

if new behavior is introduced.

- Code examples in the AIP **MUST** match the OpenAPI spec (Doc 08), JSON Schemas (Doc 10),

and SDK Reference (Doc 11). Drift is a blocking review issue.

7. Community Contributions & RFC Process

1. **Idea / RFC**. Open a Discussion describing the problem and proposed direction. Gather

early feedback (the lightweight "RFC" stage).

1. **Draft AIP**. Fork the AIP template, fill all sections, open a PR. An Editor assigns

the next AIP number.

1. **Review** → **Last Call** → **Accepted** → **Final**. Per §3.
2. **Contributor License**. Contributions are licensed Apache-2.0 (Doc 13 §code style /

Doc 15 contribution guide). A DCO sign-off is required.

8. AIP Template

```

---
aip: <assigned by editors>
title: <concise title>
author: <name / handle / email>
type: Standards Track | Meta | Informational
category: Core | Interface | Networks | Security # Standards Track only
status: Draft
created: YYYY-MM-DD
requires: <AIP numbers, optional>
supersedes: <AIP number, optional>
---

# Abstract
# Motivation
# Specification # normative; MUST/SHOULD/MAY (RFC 2119)
# Backward Compatibility
# Reference Implementation
# Security Considerations
# Test Cases
# Copyright # Apache-2.0

```

9. Example AIPs

AIP-1 — AIFP Improvement Proposal Process *(Meta, Final)*

This document. Defines the process by which all other AIPs are created and ratified.

AIP-2 — Core Payment Protocol *(Standards Track · Core, Final)*

Abstract. Ratifies AIFP-1 (Doc 01) as the normative protocol: HTTP-402 challenge, quote/pay/receipt loop, Ed25519 stateless receipts (TTL 600s), agent action pricing tiers (Standard from \$0.00001 / Complex from \$0.00006 / Premium from \$0.00010), 1% AiFinPay protocol fee, merchant settlement of the remaining 99% excluding applicable network or settlement costs, idempotency (24h), and error registry. **Backward Compatibility:** baseline; nothing to break.

AIP-7 — Add Unichain to Full Core Networks *(Standards Track · Networks, Final)*

Abstract. Adds Unichain to the Full Core tier (Core + Passport + mSECCO + Pyth), bringing supported networks to 12. **Spec:** add `unichain` to the `chain` enum (Doc 08/10), publish payout-address format, extend conformance vectors. ****Backward Compatibility:**** MINOR — new optional chain; existing clients unaffected.

AIP-12 — Agent Passport & Reputation Network *(Standards Track · Core, Final)*

Abstract. Introduces `pp_*` Agent Passports with Ed25519 identity, reputation

∈ [0,1000] (start 500), risk ∈ [0,100], and trust levels

(untrusted/basic/verified/enterprise). Adds `POST /v1/passports` , `GET /v1/passports/{id}` .

Backward Compatibility: MINOR — additive endpoints and optional challenge fields.

AIP-19 — AIFP-403-BUDGET-EXCEEDED Error *(Standards Track · Interface, Final)*

Abstract. Adds a dedicated error for budget-policy breaches at `POST /v1/pay` .

Spec: new code in the registry (Doc 01 App. C / Doc 08). **Backward Compatibility:**

MINOR — new error code under existing `403` semantics.

AIP-23 — Streaming Payments via mSECCO Channels *(Standards Track · Core, Accepted)*

Abstract. Defines payment channels for high-frequency metered consumption with escrow

bound to the Agent Passport. **Status:** Accepted; awaiting reference implementation +

conformance vectors before Final. **Backward Compatibility:** MINOR — opt-in capability

gated by Protocol Negotiation.

AIP-31 — Dynamic Pricing Reputation Discount Cap *(Informational)*

Abstract. Recommends clamping dynamic prices to `[min,max]` and capping

reputation-based discounts at **-30%** to prevent gaming. Non-binding guidance for

merchants implementing the Dynamic Pricing Engine.

10. Copyright

This document and all AIPs are released under **Apache-2.0**.